AUTO AUDITORÍA

Documentación técnica

Sistema de auditoría configurable para PostgreSQL

Modelo entidad-relación · Diccionario de datos · Configuración · Instalación y triggers

Control del documento

Campo	Detalle
Nombre	Auto Auditoría — Documentación técnica
Base de datos	control_cambios_db
Esquema del sistema	trazabilidad
Motor objetivo	PostgreSQL 18
Aplicación	Interfaz local Flask; cliente SQL DBeaver
Alcance	Configuración de tablas/columnas, registro de cambios y consulta histórica

Índice

Control del documento 2
Índice 2
1. Propósito y arquitectura 3
2. Modelo entidad-relación 3
3. Diccionario de datos 3
trz_tabla_origen 3
trz_columna_origen 3
trz_tipo_operacion 3
trz_regla_auditoria 3
trz_usuario 3
trz_estacion 3
trz_evento 3
trz_cambio_columna 3
Tabla de demostración: public.clientes_demo 3
Índices 3
4. Configurar y activar auditoría por tabla 3
4.1 Requisitos 3
4.2 Secuencia desde la interfaz 3
4.3 Ejemplo SQL para una columna 3
4.4 Desactivar una operación o una tabla 3
5. Instalación y despliegue de triggers 3
5.1 Instalación limpia 3
5.2 Cómo se generan los triggers 3
5.3 Verificación posterior 3
6. Validación, seguridad y solución de problemas 3
Controles de seguridad 3
Notas de versión y compatibilidad 3

1. Propósito y arquitectura

Auto Auditoría mantiene una configuración por tabla, columna y operación. Una función instala un trigger por tabla de origen; el trigger registra eventos y detalles en tablas centrales del esquema trazabilidad. La bitácora conserva snapshots JSONB para que el historial sobreviva a cambios o eliminación de la fila original.

Capa	Responsabilidad
Interfaz Flask	Permite operar la tabla de demostración, consultar el historial y administrar configuración según el perfil web.
Funciones PL/pgSQL	Sincronizan el catálogo, validan configuración e instalan/desinstalan triggers de auditoría.
Esquema trazabilidad	Guarda metadatos, reglas, eventos, usuarios, estaciones y detalles de columnas.
Tablas de negocio	Tablas de origen seleccionadas; la demostración usa public.clientes_demo.

Los perfiles de la interfaz son independientes de los roles PostgreSQL: operador (clientes demo), auditor (demo e historial) y administrador (además configura auditoría). La cuenta PostgreSQL de la aplicación debe tener solo los permisos que necesita; las tareas administrativas usan autoaudit_admin.

2. Modelo entidad-relación

Las relaciones 1:N se implementan con claves foráneas. Una regla une una columna con un tipo de operación; por ello materializa la configuración de qué auditar. Los datos de origen se describen mediante esquema y nombre, sin FK hacia tablas de negocio.

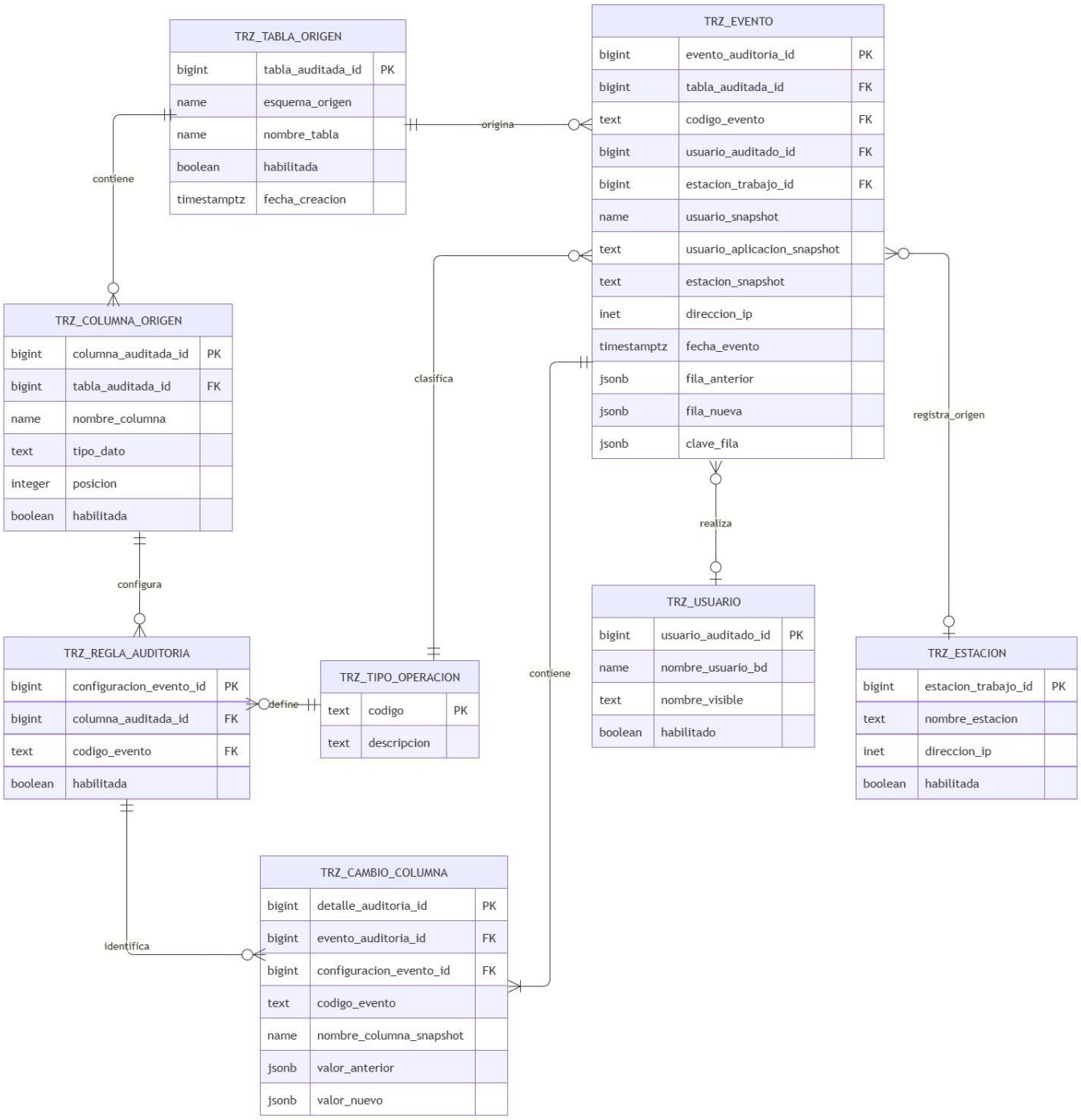


Figura 1. Entidades del esquema trazabilidad y sus relaciones principales. La tabla public.clientes_demo es una tabla de origen de ejemplo y no pertenece al esquema central.

3. Diccionario de datos

Tipos y nulabilidad indicados para la versión del proyecto. PK = clave primaria; FK = clave foránea; UNIQUE = unicidad; CHECK = regla de validación. Las columnas snapshot preservan el dato registrado, aunque se desactive o cambie la configuración posteriormente.

trz_tabla_origen

Registro de tablas que pueden auditarse.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
tabla_auditada_id	bigint	No	PK, identity	Identificador interno de tabla.
esquema_origen	name	No	UNIQUE junto a nombre_tabla	Esquema PostgreSQL de la tabla fuente.
nombre_tabla	name	No	UNIQUE junto a esquema_origen	Nombre de tabla fuente.
habilitada	boolean	No	DEFAULT false	Indica que la auditoría de la tabla está activa.
fecha_creacion	timestamptz	No	DEFAULT current_timestamp	Fecha de incorporación al catálogo.

trz_columna_origen

Columnas detectadas en las tablas de origen.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
columna_auditada_id	bigint	No	PK, identity	Identificador interno de columna.
tabla_auditada_id	bigint	No	FK -> trz_tabla_origen; RESTRICT	Tabla a la que pertenece.
nombre_columna	name	No	UNIQUE con tabla_auditada_id	Nombre exacto de la columna.
tipo_dato	text	Sí	—	Tipo PostgreSQL informado por el diccionario.
posicion	integer	Sí	CHECK NULL o > 0	Posición ordinal de columna.
habilitada	boolean	No	DEFAULT false	Indica si la columna puede generar detalle.

trz_tipo_operacion

Catálogo de operaciones auditables.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
codigo	text	No	PK; CHECK insert/update/delete	Código normalizado de operación.
descripcion	text	No	—	Descripción legible de la operación.

trz_regla_auditoria

Regla por columna y tipo de operación.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
configuracion_evento_id	bigint	No	PK, identity	Identificador de regla.
columna_auditada_id	bigint	No	FK -> trz_columna_origen; RESTRICT	Columna configurada.
codigo_evento	text	No	FK -> trz_tipo_operacion	Operación a registrar.
habilitada	boolean	No	DEFAULT false	Estado de la regla.
Restricción de tabla	Restricción de tabla	Restricción de tabla	UNIQUE (columna_auditada_id ,codigo_evento)	Evita duplicar una operación para la columna.

trz_usuario

Usuarios PostgreSQL que aparecen en auditoría.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
usuario_auditado_id	bigint	No	PK, identity	Identificador de usuario.
nombre_usuario_bd	name	No	UNIQUE	Nombre de rol/login PostgreSQL asociado.
nombre_visible	text	Sí	—	Nombre descriptivo opcional.
habilitado	boolean	No	DEFAULT true	Permite usar este mapeo de usuario.

trz_estacion

Estaciones o clientes identificados en la auditoría.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
estacion_trabajo_id	bigint	No	PK, identity	Identificador de estación.
nombre_estacion	text	No	UNIQUE	Nombre de estación reportado.
direccion_ip	inet	Sí	—	Dirección de red asociada, si está disponible.
habilitada	boolean	No	DEFAULT true	Indica si la estación está activa.

trz_evento

Cabecera de cada operación ejecutada sobre una tabla auditada.

Columna	Tipo	Nulo	Restricción / referencia	Descripción

evento_auditoria_id	bigint	No	PK, identity	Identificador de evento.
tabla_auditada_id	bigint	No	FK -> trz_tabla_origen	Tabla origen del evento.
codigo_evento	text	No	FK -> trz_tipo_operacion	insert, update o delete.
usuario_auditado_id	bigint	Sí	FK -> trz_usuario; SET NULL	Usuario mapeado si existe.
estacion_trabajo_id	bigint	Sí	FK -> trz_estacion; SET NULL	Estación mapeada, si existe.
usuario_snapshot	name	No	DEFAULT session_user	Usuario PostgreSQL preservado en el evento.
usuario_aplicacion_snapshot	text	Sí	Columna añadida por migración 09	Usuario de interfaz enviado por la app si la función 10 está instalada.
estacion_snapshot	text	Sí	—	Nombre reportado de estación.
direccion_ip	inet	Sí	—	IP cliente, si PostgreSQL la conoce.
fecha_evento	timestamptz	No	DEFAULT clock_timestamp()	Fecha y hora del evento.
fila_anterior	jsonb	Sí	CHECK según operación	Snapshot completo anterior para update/delete.
fila_nueva	jsonb	Sí	CHECK según operación	Snapshot completo nuevo para insert/update.
clave_fila	jsonb	Sí	—	Clave de la fila para localizar el objeto lógico.

trz_cambio_columna

Detalle de cada columna configurada que cambió.

Columna	Tipo	Nulo	Restricción / referencia	Descripción
detalle_auditoria_id	bigint	No	PK, identity	Identificador del detalle.
evento_auditoria_id	bigint	No	FK compuesta con codigo_evento	Evento al que pertenece.
configuracion_evento_id	bigint	No	FK compuesta con codigo_evento	Regla que habilitó el detalle.
codigo_evento	text	No	Parte de ambas FK compuestas	Operación capturada.
nombre_columna_snapshot	name	No	—	Nombre de columna preservado al registrar.
valor_anterior	jsonb	Sí	—	Valor previo; NULL para insert.
valor_nuevo	jsonb	Sí	—	Valor nuevo; NULL para delete.

Tabla de demostración: public.clientes_demo

Columna	Tipo	Nulo	Restricción	Descripción

id	bigint	No	PK, identity	Identificador del cliente.
nombre	text	No	—	Nombre de demostración.
correo	text	No	UNIQUE	Correo; habilitado para INSERT/UPDATE/DELETE.
ciudad	text	Sí	—	Ciudad; habilitada para las tres operaciones.

Índices

Tabla	Índice	Uso
trz_tabla_origen	UNIQUE (esquema_origen,nombre_tabla)	Evita duplicar tablas.
trz_columna_origen	UNIQUE (tabla_auditada_id,nombre_columna)	Evita duplicar columnas por tabla.
trz_regla_auditoria	UNIQUE (columna_auditada_id,codigo_evento)	Una regla por operación y columna.
trz_evento	fecha desc; tabla+fecha; usuario+fecha	Facilitan filtros usuales del historial.
trz_cambio_columna	configuracion_evento_id; nombre_columna_snapshot+evento_auditoria_id	Filtrado por columna y regla.

4. Configurar y activar auditoría por tabla

4.1 Requisitos

- La tabla ya existe y el usuario PostgreSQL puede verla y consultar sus columnas.
- El esquema trazabilidad, las tablas y funciones han sido instalados.
- La cuenta que configura pertenece a autoaudit_admin y puede ejecutar las funciones administrativas.
- Las reglas se aplican solo a columnas seleccionadas; una actualización solo registra columnas cuyo valor realmente cambió.

4.2 Secuencia desde la interfaz

1. Inicia sesión con el perfil administrador.
2. Abre Configuración de auditoría y usa Sincronizar diccionario para descubrir tablas y columnas visibles.
3. Selecciona esquema, tabla y columnas a auditar. Marca INSERT, UPDATE y/o DELETE para cada columna.
4. Guarda la configuración. La aplicación llama la función trz_configurar_columna, que valida metadatos y mantiene el trigger de esa tabla.
5. Regresa a Clientes demo o ejecuta una operación sobre la tabla. Abre Historial de control de cambios y filtra por esquema, tabla, usuario, columna u operación.

4.3 Ejemplo SQL para una columna

Ejecutar conectado a control_cambios_db con un usuario autorizado. La aplicación administrativa es el camino preferido; el ejemplo muestra la función utilizada.

```
SELECT trazabilidad.trz_sincronizar_diccionario();

SELECT trazabilidad.trz_configurar_columna(
  'public', 'clientes_demo', 'correo',
  ARRAY['insert','update','delete']::text[]
);
```

4.4 Desactivar una operación o una tabla

```
SELECT trazabilidad.trz_desactivar_operacion(
  'public', 'clientes_demo', 'correo', 'delete'
);

SELECT trazabilidad.trz_desactivar_tabla('public', 'clientes_demo');
```

Confirma las firmas exactas en la versión instalada si la clase modificó los parámetros. Desactivar una regla conserva el historial previo; no borra eventos existentes. Desactivar una tabla puede retirar su trigger de captura.

5. Instalación y despliegue de triggers

5.1 Instalación limpia

Realiza respaldo antes de instalar. Con DBeaver, selecciona explícitamente la conexión y base correctas. El script 00 crea la base desde una conexión de mantenimiento; los demás se ejecutan ya conectado a control_cambios_db.

6. 00_crear_base.sql: ejecutar conectado a postgres (o base de mantenimiento). Si la base ya existe, no recrearla sin respaldo.
7. Conectar a control_cambios_db y confirmar SELECT current_database(), current_user;.
8. 01_crear_schema.sql: crea trazabilidad y ajusta search_path. ADVERTENCIA: esta versión elimina y vuelve a crear el esquema; hacerlo borra sus objetos y bitácora.
9. 02_esquema_tablas.sql: crea tablas, claves, restricciones e índices centrales.
10. 03_funciones.sql: instala funciones de sincronización, configuración, trigger y consulta.
11. 09_contexto_usuario_web.sql y 10_actualizar_funcion_usuario_web.sql: ejecutar en ese orden cuando se vaya a registrar el usuario de interfaz; son migraciones de la versión web. En una instalación distinta, comprueba que estén incluidos.
12. 04_permisos.sql: crea/ajusta el rol autoaudit_admin y permisos de funciones. Concede membresía a cuentas autorizadas; la app debe recibir solo los privilegios requeridos.
13. 05_configuracion_demo.sql: instala public.clientes_demo, sincroniza metadatos, crea el trigger de contexto en trz_evento y configura sus tres columnas. ADVERTENCIA: elimina y recrea clientes_demo.
14. 06_consultas_historial.sql: prepara vistas y consultas de historial.
15. 07_pruebas_funcionales.sql y 08_pruebas_rendimiento.sql: usar en entorno de validación; leer comentarios y ejecutar por bloques según indique cada archivo.

5.2 Cómo se generan los triggers

16. Al activar la primera regla de una tabla, trz_configurar_columna valida que esquema, tabla y columna existan.
17. La función actualiza el catálogo y delega en trz_instalar_trigger la creación del trigger específico para la tabla origen.
18. El trigger de fila AFTER INSERT OR UPDATE OR DELETE ejecuta trz_registrar_cambio. Cada trigger maneja una sola tabla.
19. trz_registrar_cambio consulta las reglas activas, almacena una cabecera en trz_evento y agrega una fila en trz_cambio_columna por cada columna aplicable.
20. En UPDATE no genera detalle por una columna configurada si OLD y NEW son iguales.
21. El trigger trg_trz_contexto se ejecuta BEFORE INSERT sobre trazabilidad.trz_evento y completa contexto disponible (por ejemplo IP/estación).
22. Los triggers corren dentro de la transacción de la operación fuente: si esta hace ROLLBACK, el evento también se revierte.

5.3 Verificación posterior

```
SELECT current_database(), current_schema(), current_setting('search_path');

SELECT schemaname, tablename, trigger_name, action_timing, event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'public'
  AND event_object_table = 'clientes_demo'
ORDER BY trigger_name, event_manipulation;
```

Prueba INSERT, UPDATE de un valor, UPDATE sin cambio en columna auditada y DELETE. Consulta el historial por correo/clave y verifica usuario, operación, fecha, snapshots y valores anteriores/nuevo. Los scripts funcionales que usan ROLLBACK validan sin dejar filas persistentes.

6. Validación, seguridad y solución de problemas

Síntoma	Causa probable	Qué revisar
No aparecen tablas en configuración	Diccionario sin sincronizar, conexión a otra base o falta de permisos SELECT sobre la tabla.	current_database(); roles/permisos; ejecutar sincronización con el perfil administrador.
No se crea trigger	Reglas deshabilitadas, tabla/columna no visible, o configuración guardada en otra conexión.	Estado en trz_tabla_origen, trz_columna_origen y trz_regla_auditoria; errores devueltos por la función.
Historial vacío	La operación no ocurrió, el trigger no existe o la transacción se revirtió.	Revisar trigger, reglas activas y ejecutar una operación confirmada; el test con ROLLBACK no deja datos.
SQLSTATE 25P02	Una sentencia anterior falló dentro de la transacción.	En DBeaver hacer ROLLBACK o terminar la transacción fallida, corregir el primer error y volver a ejecutar.
No existe relation/function	Conexión a base incorrecta o scripts faltantes/fuera de orden.	Verificar base/esquema y repetir instalación en el orden documentado; no ejecutar comandos sobre postgres por error.

Controles de seguridad

- Solo administradores deben configurar tablas y columnas.
- Evita escritura directa a las tablas de configuración y bitácora; usa funciones autorizadas.
- Conserva acceso de escritura a la bitácora únicamente para funciones/triggers bajo el rol propietario previsto.
- Restringe acceso a snapshots: pueden contener datos personales o confidenciales.
- La IP y el usuario de aplicación se registran según las políticas de privacidad y retención de la organización.
- Guarda .env fuera del repositorio; publica solo .env.example con valores de muestra.

Notas de versión y compatibilidad

El documento refleja el diseño del proyecto descrito: base control_cambios_db, esquema trazabilidad y nombres trz_*. Antes de entregar, compara nombres, tipos y firmas con los scripts finales efectivamente subidos al repositorio. Las migraciones 09/10 pertenecen a la ampliación de contexto de usuario web; omítelas solo si la instalación no usa ese dato y el SQL fue ajustado de forma coherente.